Bearer Token: del riesgo al sender-constrained

25 minutos, tres actos



El billete al portador

- **Posesión = autorización:** el token se presenta y se acepta.
- **No se verifica el dispositivo** ni el cliente que lo usa.
- **Si se filtra:** quien lo tiene, lo usa.

Fuente: RFC 6749, Bearer Token Usage (octubre 2012).

El robo del token

- **Facebook (2018):** 50 millones de tokens robados vía «Ver como».
- **Starbucks (2019):** claves API expuestas en GitHub.
- **En ambos:** el token se usó sin restricciones.

Fuente: Meta Security Report (2018); HackerOne (2019).

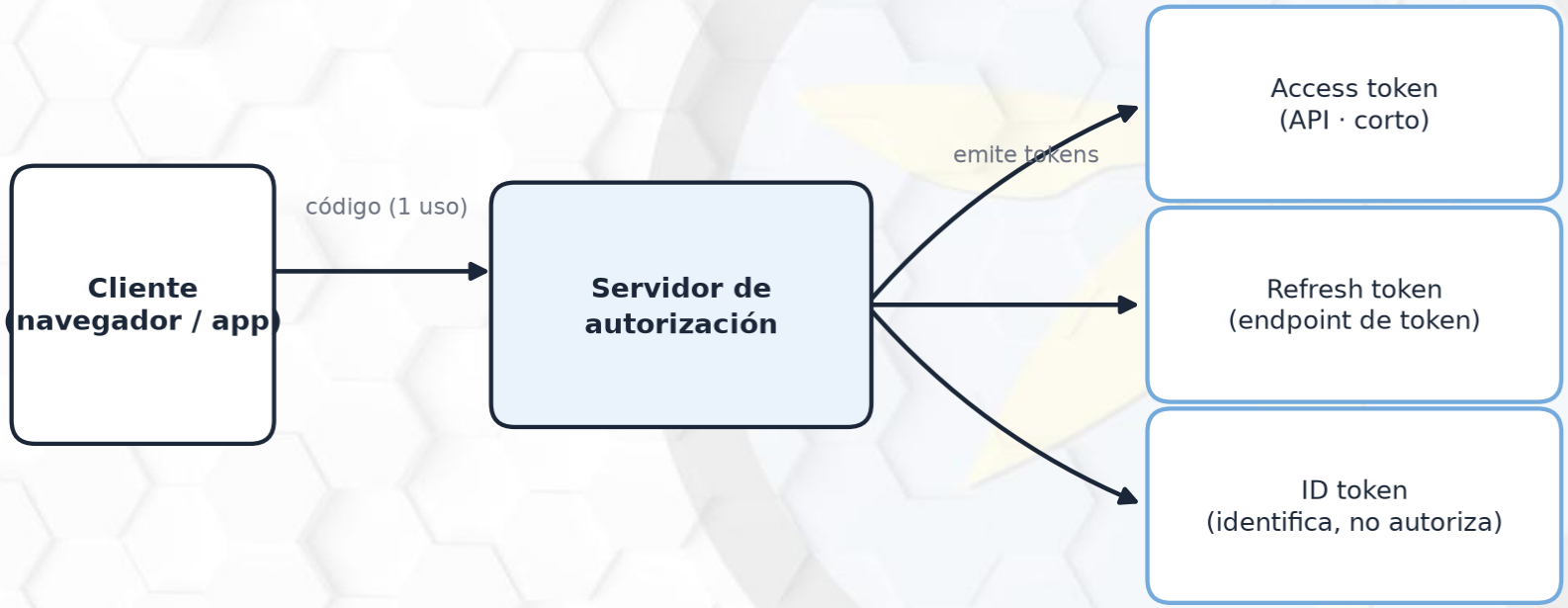
Qué no cubre el bearer

- **No cifra:** JWT es base64, no cifrado.
- **No autoriza:** es otro capítulo.
- **No previene replay:** interceptado = reutilizado.
- **No verifica dispositivo** ni cliente.
- **No es MFA:** la autenticación ya pasó.

Fuente: RFC 6749 §10.3 (2012).

LÍNEA DE TIEMPO

De Basic al sender-constrained



Fuente: RFC 2617 (1999); RFC 6749 (2012); RFC 8705 (2020); RFC 9449 (2023).

Seguridad dinámica: Step-Up

- **La autenticación no es binaria:** es un flujo continuo.
- **Operaciones sensibles:** exigen re-autenticación.
- **Mecanismo:** RFC 9470, Step-Up Authentication Challenge.

Fuente: RFC 9470 (2023); NIST SP 800-63-4 (2025).

Step-Up: flujo 401 y reintento

- **1. La API evalúa:** transacción sensible, acr insuficiente.
- **2. Responde 401:** WWW-Authenticate con acr_values.
- **3. El frontend redirige al IdP** para segundo factor.
- **4. Nuevo token con acr actualizado.** Reintento exitoso.

Fuente: RFC 9470 (2023).

Claims acr y amr

- **acr:** Authentication Context Class Reference.
- **amr:** Authentication Methods References.
- **Dicen cómo se autenticó,** no solo quién.

Fuente: RFC 8176 (2017); OIDC Core (2014).

M2M: sin factorizar

- **El invocador es un microservicio** o job de CI.
- **No hay segundo factor:** el equivalente es mTLS.
- **Certificados X.509** o Private Key JWT para firmado de peticiones.

Fuente: RFC 8705 (2020); RFC 7523 (2015).

Apps modificadas

- **Las apps están en manos del atacante:** modificables, descompilables.
- **Riesgo:** cliente Frankenstein que simula ser la app legítima.
- **Desactivan SSL pinning,** extraen secretos, llaman a la API.

Fuente: OWASP Mobile Top 10 (2024).

PKCE: candado del código

- **code_verifier:** string aleatorio generado por el cliente.
- **code_challenge:** hash del verifier, enviado al pedir el código.
- **Al canjear:** el verifier prueba la posesión.
- **Obligatorio hoy** en clientes públicos (RFC 9700).

Fuente: RFC 7636 (2015); RFC 9700 (2025).

App Attestation

- **Android:** Play Integrity API.
- **iOS:** DeviceCheck / App Attest.
- **Verifica el binario no fue modificado** y emite token de attestation.
- **La API lo verifica** antes de operaciones sensibles.

Fuente: Google Play Integrity; Apple DeviceCheck.

Mobile: tres capas

- **App Attestation:** el binario es legítimo.
- **PKCE:** el código no se puede reutilizar.
- **DPoP:** el token no sirve en otro dispositivo.
- **Cliente Frankenstein:** sin binario legítimo, sin PKCE, sin DPoP.

Fuente: Google; Apple; RFC 7636; RFC 9449.

COMPARATIVA

DPoP vs mTLS

Característica	mTLS	DPoP
Capa OSI	Capa 4 (Transporte)	Capa 7 (Aplicación)
Mecanismo	Certificado X.509 en TLS	JWT firmado en header HTTP
Gestión de claves	Compleja (PKI)	Sin PKI, par dinámico
Ideal para	M2M, microservicios	SPAs, móviles

Fuente: RFC 8705 (2020); RFC 9449 (2023).

DPoP: prueba en HTTP

- **Par de claves dinámico:** generado en el momento.
- **Token atado a la clave pública.** La privada permanece en el dispositivo.
- **Header DPoP en cada request:** JWT firmado con método y URL.
- **Funciona a través de proxies, WAFs** y terminaciones TLS.

Fuente: RFC 9449 (2023).

COMPARATIVA

DPoP vs mTLS: resumen

- **mTLS:** capa 4, certificados, PKI, ideal M2M.
- **DPoP:** capa 7, headers HTTP, ideal clientes públicos.
- **Ambos atan el token** al cliente legítimo.
- **Robar el token** sin la prueba criptográfica: inútil.

Fuente: RFC 8705 (2020); RFC 9449 (2023).

TLS, Gateway, Zero Trust



Fuente: OWASP API Security Top 10 (2023); NIST SP 800-207 (2020).

Canal seguro: límites

- **TLS:** protege en tránsito, no en el cliente.
- **Gateway:** throttling, WAF, validación de firma, revocación.
- **Error común:** microservicios confían ciegamente.
- **Zero Trust:** propagar claims, autorizar en cada capa.

Fuente: OWASP API Top 10 (2023); NIST SP 800-207 (2020).

ID Uruguay

- **OIDC con bearer tokens:** tokens de larga vida sin rotación.
- **SPAs con token en localStorage:** robo por XSS.
- **Mitigaciones:** DPoP, PKCE, Step-Up para operaciones sensibles.

Fuente: AGESIC / ID Uruguay; OWASP.

Tres acciones

- **Uno:** auditar la vida de los tokens.
- **Dos:** revisar almacenamiento en clientes.
- **Tres:** identificar operaciones sensibles para Step-Up.

Fuente: OWASP API Top 10 (2023); RFC 9700 (2025).

CIERRE

El token se acaba. El riesgo no.

- **Autenticar es la puerta.** Autorizar es la cerradura.
- **El token bearer es el presente.** El sender-constrained es el siguiente paso.
- **La seguridad no está en el string:** está en el sistema.

Fuente: OWASP; RFC 9449; RFC 9700.